Rajat Gupta (P25AI0008)

Artificial Intelligence, Assignment 2026

November 2026

1 Introduction

The task is to decide logical entailment: given a knowledge base (KB) of facts and rules and a query, is the query a logical consequence of the KB? The output is binary, so a query that is neither provable nor refutable (*Unknown*) counts as `False`. The assignment requires classical AI methods implemented in Python; no generative model may produce an answer. The evaluators confirmed that the private test supplies the logic directly rather than natural-language sentences. Its notation, its column names and its world assumption (closed or open) are not specified, and no item is tagged with the dataset it comes from.

The design follows from these constraints. A tolerant parser accepts every common notation for first-order logic and maps it to one internal representation. Several classical engines then work on that representation, and a router decides, from the structure of the knowledge base alone, which engine and which semantics apply. Figure 1 shows the complete pipeline.

2 Data and how it is used

2.1 The three development datasets

- **RuleTaker** contains facts and rules over a small vocabulary ("Bob is young", "If someone is kind and young then they are not smart"), grouped in six proof-depth configurations (0, 1, 2, 3, 3ext, 5). Labels follow the closed-world assumption: a negated question about an attribute that never occurs in the context is labelled true (302 of 302 such questions in the training data).
- **FOLIO v2** provides human-written stories with first-order annotations (`premises-FOL`, `conclusion-FOL`) using $\forall \exists \neg \wedge \vee \rightarrow \leftrightarrow \oplus$. Labels are True, False or Uncertain, and the semantics is the open world.
- **CounterBench** consists of deterministic causal graphs with nonsense variable names and counterfactual questions of five types (basic, joint, nested, conditional, backdoor). Each problem occurs five times with renamed variables.

Notebook Section 2 explores every dataset on its tune split only: label balance, depth configurations, facts and rules per context, operator frequencies, the share of FOLIO questions that cannot be written as Horn rules, and the sentence patterns of CounterBench.

2.2 Tune and report splits

Data used to make a decision is never used to report accuracy (Figure 2). RuleTaker is tuned on its dev split and reported on its test split; FOLIO is tuned on train and reported on validation; CounterBench has no official split, so it is divided by *problem*, keeping all five renamed copies of a problem on the same side, stratified by type (600 / 600). An automatic check removes from the tune side any context, story or problem that also occurs on the report side. No model is trained:

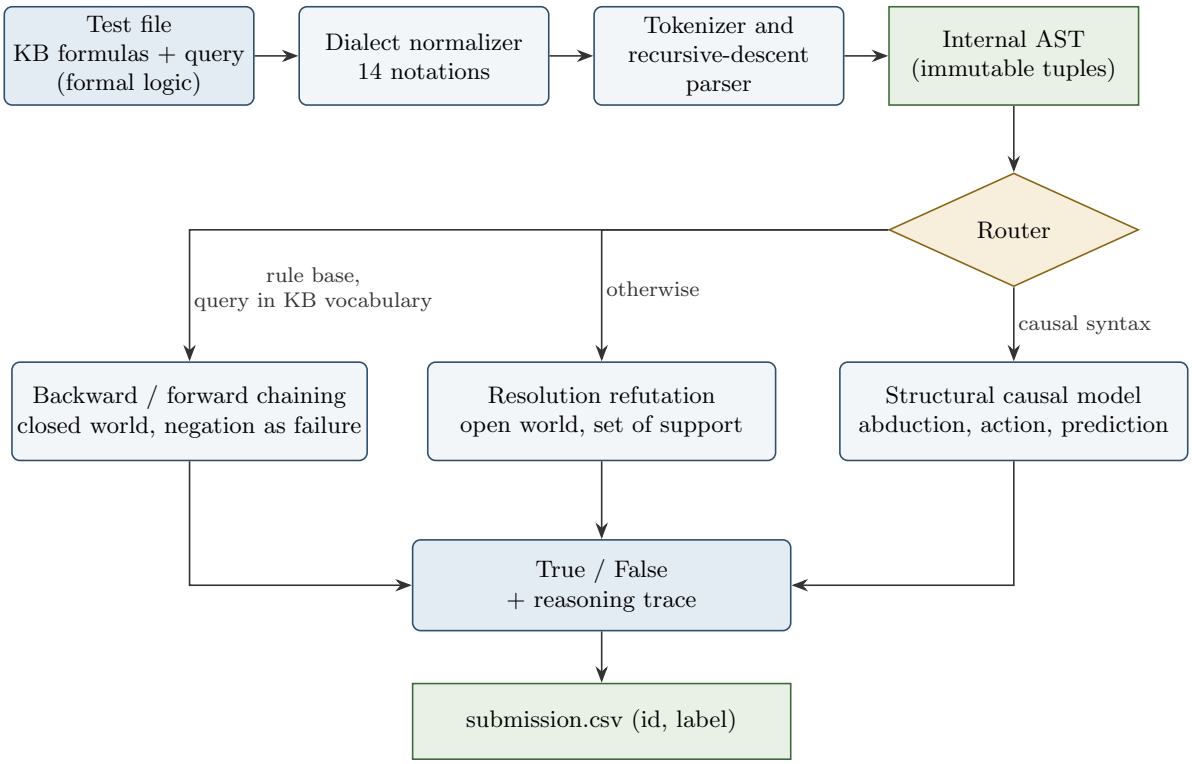


Figure 1: System pipeline. Every notation is normalised and parsed into one abstract syntax tree; the router sends each problem to exactly one engine, which returns the answer together with a reasoning trace.

the datasets serve to build and test the parser, to choose the few settings of the engines, and to measure accuracy.

2.3 Logic-only test files

Because the private test is written in logic, the notebook also answers two test files that contain only formulas. The *private test* (`private_test.csv`) is built from ProverQA, a public first-order dataset whose labels were verified with the Prover9 theorem prover; 100 problems are drawn from each difficulty level. The *public test* (`public_test.csv`) is built from the report splits: 1,200 RuleTaker questions converted to logic, all 203 FOLIO validation questions and the 600 held-out CounterBench questions written as structural equations. Both files are checked to contain no natural-language sentence before they are answered.

3 Representation

Every input is parsed into one immutable, hashable structure made of Python tuples (Table 1). Facts and clauses can therefore be kept in sets and dictionaries, which gives constant-time membership tests, automatic removal of duplicates and safe caching.

Parsing. Parsing happens in three stages. A *dialect normalizer* rewrites foreign notations into a canonical infix form; each rewrite fires only on its own markers, so a rule for one dialect cannot damage another. A *tokenizer* maps every spelling of an operator to one token ($\rightarrow$, `->`, `=>`, $\Rightarrow$, $\supset$, `implies` all become IMP). A *recursive-descent parser* implements the precedence $\leftrightarrow < \rightarrow < \oplus < \vee < \wedge < \neg$, with $\rightarrow$ right-associative. Table 2 lists the fourteen notations that are accepted.

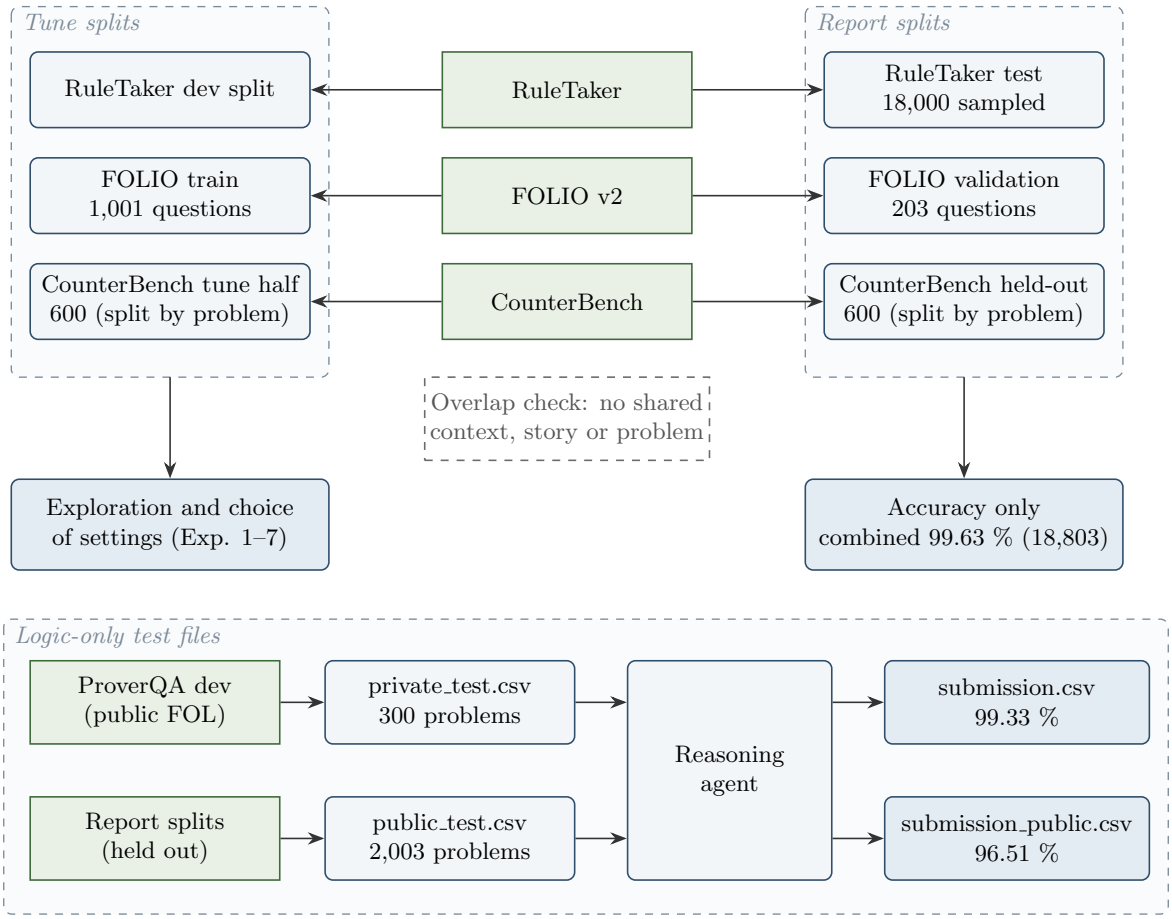


Figure 2: Data discipline. Tune splits are used for exploration and for every design choice; report splits are used only to measure accuracy. Two logic-only test files are answered by the final agent.

Variables and scope. Quantified variables are renamed apart. Free variables x, y, z in the knowledge base (and every capitalised argument in Prolog or TPTP input) are universally quantified. A free variable in a query is read existentially when the query is a combination of literals ("is there such an x?", as in a Prolog query) and universally when the query is rule-like. The scope of "$\forall x\,(\dots)$" is the bracket unless x is used after it. Constants and predicate names are case-insensitive.

Fault tolerance. Parsing is strict first; only if it fails is a tolerant mode used (a missing operator is read as $\wedge$, a single unbalanced bracket is repaired), and its use is counted. An unreadable formula is dropped and reported instead of discarding the whole knowledge base. Of the 7,597 FOLIO formulas, 7 fail to parse; all of them are genuinely broken annotations.

4 Reasoning engines

4.1 Unification

The most general unifier is computed with an occurs check. Substitutions are dictionaries copied on write, so a failed attempt never corrupts the state of a proof.

4.2 Forward chaining

Facts are matched against rule bodies until a fixpoint is reached. Negation as failure in rule bodies is made sound by *stratification*: a predicate that is used negatively is completed before it is tested. Variables that are not bound by a positive literal range over the Herbrand domain, which is

Object	Python form	Example
Variable	string starting with `?`	`?x1` (unique per quantifier)
Constant	lower-case string	`bob`, `captain_america`
Function / Skolem term	`(name, (args))`	`('sk1', ('?x1',))`
Atom	`('atom', pred, args)`	`('atom', 'Young', ('bob',))`
Formula	nested tuples: `not, and, or, imp, iff, xor, forall, exists`	`('imp', A, B)`
Literal	`(positive?, pred, args)`	`(False, 'Smart', ('bob',))`
Clause (resolution)	frozenset of literals	$\{\neg Young(?0),\ Smart(?0)\}$
Rule (chaining)	`(body literals, head)`	`([Young(?x)], Smart(?x))`
Structural equation	{`variable: formula`}	$Blaf := \neg Ziklo$

Table 1: Internal representation.

Notation	Example of the same rule or fact
Unicode FOL (FOLIO)	`∀x (Young(x) → Smart(x))`
ASCII and keywords	`forall x (Young(x) -> Smart(x))`, `young(x) implies smart(x)`
Prover9	`all x (Young(x) -> Smart(x)).`
Prolog / ProbLog	`smart(X) :- young(X).  0.9::young(bob).`
TPTP	`fof(r1, axiom, ![X]: (young(X) => smart(X))).`
LaTeX	`\forall x (Young(x) \rightarrow Smart(x))`
Textbook symbols	`(∀x)(Young(x) ⊃ Smart(x))`
RuleTaker triples	`("Bob" "is" "young" "+")`
FLD	`(x): {A}x -> {B}x`
z3py	`ForAll(x, Implies(Young(x), Smart(x)))`
Logic-LM / Pyke	`Young($x, True) >>> Smart($x, True)`
SMT-LIB	`(assert (forall ((x U)) (=> (Young x) (Smart x))))`
Lean	`∀ x, Young x → Smart x`
Set notation	`Young ⊆ Smart,  bob ∈ Young`

Table 2: Accepted notations, shown on the same rule.

extended when the query names a new constant. Every derived fact stores its justification, which yields the proof trees of Section 9.

4.3 Backward chaining

Goals are proved top-down with memoisation and loop detection (a goal that is being proved cannot be used to prove itself). The semantics of negation as failure is the same as in forward chaining, but backward chaining is also correct on rule bases that are not stratified, and it explores only the goals that the query needs.

4.4 Resolution refutation

$KB \models q$ is shown by deriving the empty clause from $KB \wedge \neg q$ (Figure 3). The clause form removes $\rightarrow$, $\leftrightarrow$ and $\oplus$, builds the negation normal form, Skolemises and distributes to CNF. When distribution would produce more than 64 clauses, a Tseitin-style definition replaces the sub-formula; this preserves satisfiability, so refutations stay sound. The given-clause loop uses the *set of support* strategy (only descendants of $\neg q$ are resolved), best-first selection by clause weight with every fifth pick taken by age, factoring, tautology deletion and unit subsumption. Equality axioms are added

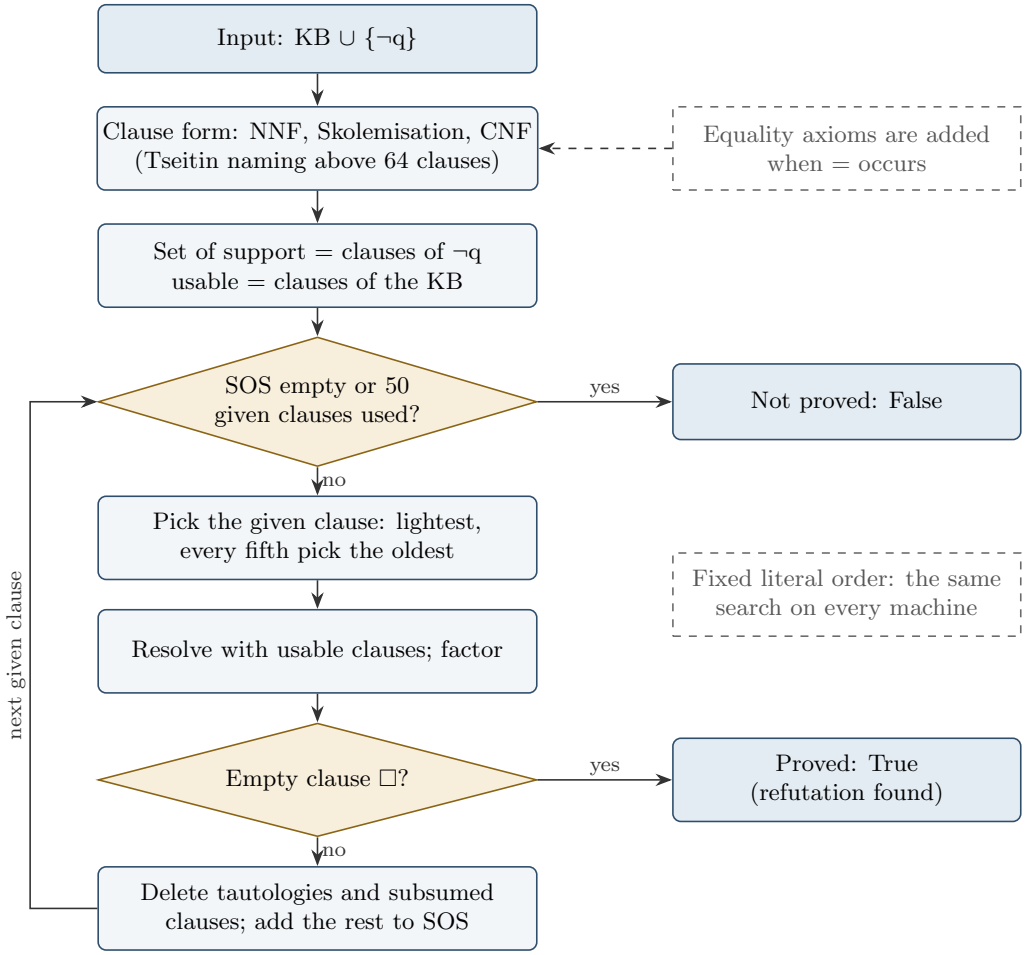


Figure 3: Given-clause loop of the resolution prover with the set-of-support strategy.

when = occurs. The budget is 50 given clauses; because the literal order is fixed, the search is identical on every machine.

4.5 Structural causal model

Causal problems are deterministic structural equations, optionally with priors $V \sim \mathrm{Bern}(p)$. A counterfactual is answered with Pearl's three steps (Figure 4): *abduction* enumerates the values of the exogenous roots that are consistent with the evidence, *action* applies $do(X{=}x)$ by removing the equations of the intervened variables, and *prediction* evaluates the modified model in topological order. The answer is the weighted share of worlds in which the query holds. For conditional queries $(Y_x \mid Z_x = z)$ the observation is applied in the post-intervention world; an observation of an intervened variable always belongs to the factual world. A cyclic graph raises `CausalCycleError`, which is reported together with the cycle.

5 Router

The router (Figure 5) chooses the engine from the structure of the input:

1. Causal syntax ($do(\cdot)$, `:=`, Y_x, $\square\!\!\rightarrow$, Bernoulli priors, "causes") sends the problem to the causal model. If causal parsing fails for a reason other than a cycle, the problem continues with the logic routes.
2. A knowledge base made only of literal facts and rules with at most one variable is a stratified logic program. If every predicate and constant of the query occurs in it, it is answered under

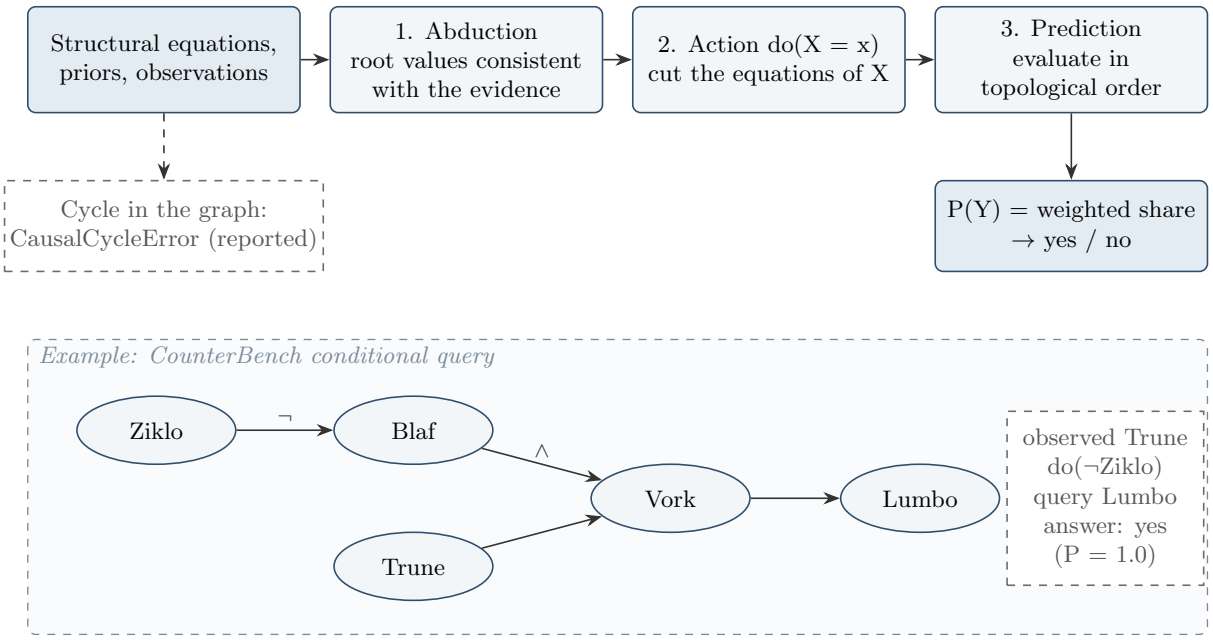


Figure 4: Counterfactual inference in the structural causal model, with the CounterBench example of Section 9.

Dataset	Split (role)	Queries	Accuracy	ms / query
RuleTaker	test (report)	18,000	100.00 %	0.69
FOLIO v2	validation (report)	203	85.22 %	8.76
CounterBench	held-out half (report)	600	93.33 %	0.15
Combined	report splits	**18,803**	**99.63 %**	
Macro average	report splits		92.85 %	
FOLIO v2	train (tune)	1,001	81.12 %	4.13
CounterBench	tune half (tune)	600	94.17 %	0.15

Table 3: Accuracy on the three development datasets. Only the report rows enter the combined accuracy.

the closed world by backward chaining. The closed world covers only the vocabulary of the knowledge base: a query about a symbol it never mentions goes to the open world.

3. A Horn knowledge base with a positive literal query is answered by forward chaining, because both worlds agree.
4. Everything else is answered under the open world by resolution refutation. Before resolution, a query symbol that does not occur in the KB is aligned to a near-identical one (similarity at least 0.85, `NameAfter` → `NamedAfter`); a word-level guard refuses changes of meaning such as Male/Female.

Any exception yields `False` and the reason is recorded.

6 Results

Accuracy is the share of correct `True`/`False` answers. Accuracies are deterministic; times are wall-clock and depend on the machine. Table 3 reports the report splits and, for reference, the tune splits; Table 4 reports the logic-only test files.

RuleTaker is answered correctly at every proof depth. The CounterBench errors are concentrated in one cyclic graph (five renamed copies), which no acyclic structural model can represent; the

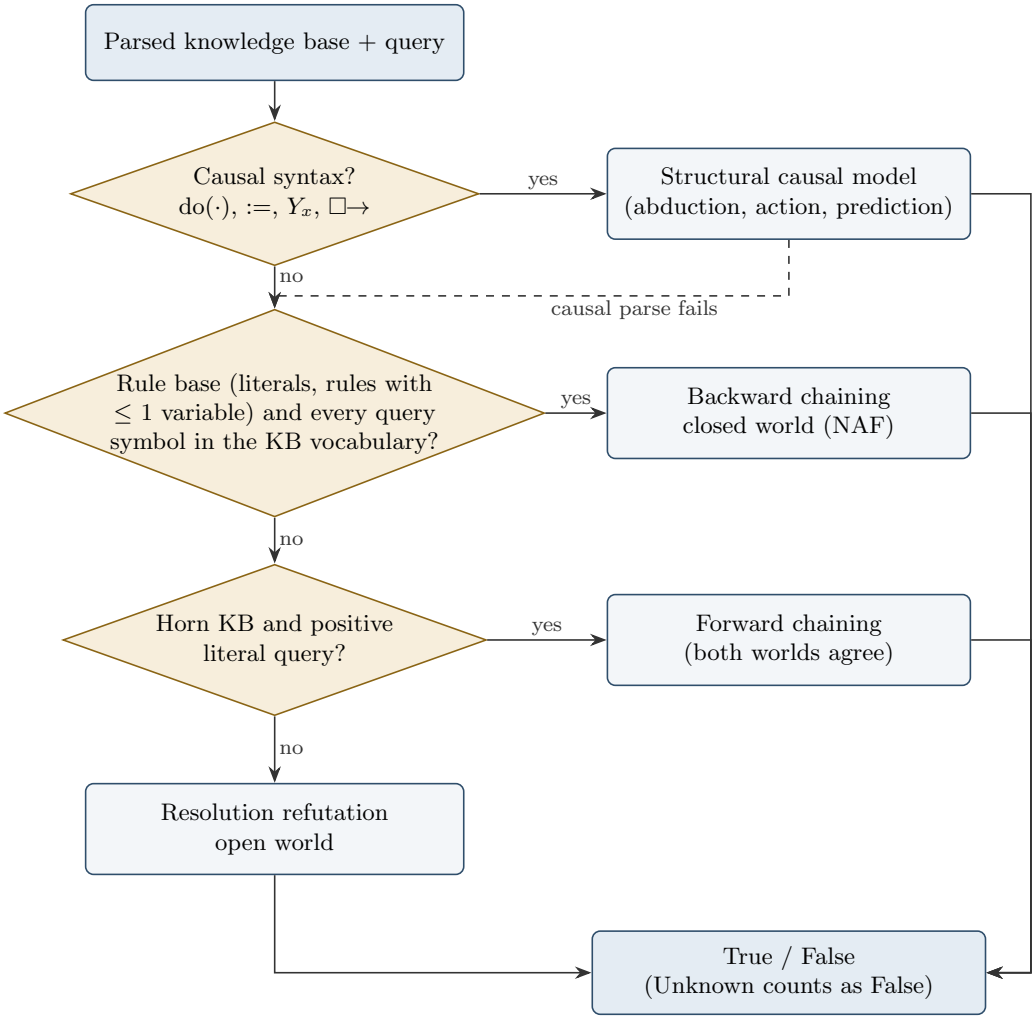


Figure 5: Router decision flow. No dataset flag is used; the decision depends only on the input.

agent reports these items instead of silently answering them.

7 Experiments

Every setting below was chosen on a tune split. Report-split values are shown only to confirm the choice.

7.1 Experiment 1: forward versus backward chaining

On 800 complete RuleTaker contexts (7,514 questions), backward chaining took 0.13 s and answered every question correctly; forward chaining took 0.53 s and missed 3, all in rule bases that are not stratified. On 2,830 such items backward chaining reached 100 % against 99.86 % for forward

Test file (logic only)	Composition	Problems	Accuracy
`private_test.csv`	ProverQA dev: easy / medium / hard easy 100.00 %, medium 98.00 %, hard 100.00 %	300	**99.33 %**
`public_test.csv`	RuleTaker 1,200 + FOLIO 203 + CounterBench 600	2,003	96.51 %
built-in test	hand-written problems, all engines	26	100.00 %

Table 4: Logic-only test files answered by the final notebook.

chaining. Backward chaining is therefore used for closed-world answers, and forward chaining only to build proof trees.

7.2 Experiment 2: closed versus open world

Semantics	RuleTaker	FOLIO
closed world (forced)	100.00 %	85.22 %
open world (forced)	74.87 %	85.22 %
automatic (router)	**100.00 %**	**85.22 %**

Forcing the open world on RuleTaker costs about 25 points, because RuleTaker's negated questions about absent attributes are true only under the closed world. The structural router matches the best fixed choice on both datasets without knowing where a problem comes from.

7.3 Experiment 3: resolution budget, query alignment and free variables

Resolution budget	Train (tune)	Validation	Train time (s)	Val. time (s)
wall clock 0.3 s (earlier)	81.12 %	85.71 %	45.06	12.26
20 given clauses	79.82 %	85.22 %	1.55	0.79
50 given clauses (chosen)	**81.12 %**	85.22 %	3.83	1.86
100 given clauses	81.12 %	85.22 %	11.21	4.82
200 given clauses	81.12 %	85.22 %	32.54	10.56

Setting	Train (tune)	Validation
query alignment off	80.62 %	84.73 %
query alignment on (threshold 0.85, word guard)	**81.12 %**	85.22 %
free query variable always $\forall$	80.92 %	84.73 %
$\exists$ **for literal queries,** $\forall$ **for rule-like queries**	**81.12 %**	85.22 %

A budget counted in given clauses makes results identical on every machine, which a wall-clock limit cannot. Fifty given clauses is the smallest budget with the best training accuracy and is 6–12 times faster than the earlier 0.3 s limit. Query alignment and the existential reading of free query variables each add about half a point.

7.4 Experiment 4: robustness to surface changes

Data	Original	Renamed symbols	ASCII syntax	Shuffled premises
RuleTaker (3,000)	100.00 %	100.00 %	100.00 %	100.00 %
FOLIO validation (203)	85.22 %	84.73 %	85.22 %	85.22 %
CounterBench (1,200)	93.75 %	93.75 %	n/a	n/a

Predicates and constants are replaced by random eight-letter words, formulas are rewritten in ASCII, or premises are shuffled. The answers do not change, so nothing depends on memorised names. The single FOLIO change under renaming is expected: random names remove the typo similarity that query alignment uses. FOLIO validation rewritten in z3py, SMT-LIB and Lean parses to identical formulas, and adding the newer notations left all 413,336 normalised formulas of the three datasets unchanged.

7.5 Experiment 5: independent truth-table oracle

Random propositional knowledge bases and queries over four atoms (depth at most 3, all connectives) are printed as text and passed through the full pipeline; a truth table over all 16 models gives the reference. The answers agree on 346 / 346 problems, which tests the parser, the clause form and the prover independently of any dataset label.

7.6 Experiment 6: choosing the counterfactual semantics

Type	Hypothesis	Tune	Held-out
conditional	observation in the factual world	36.0 %	32.0 %
conditional	**observation in the post-intervention world**	**92.0 %**	80.0 %
nested	**H1:** $do(X{=}0, V{=}0)$	**92.0 %**	96.0 %
nested	H2: $do(V{=}0)$ only	76.0 %	76.0 %
nested	H3: $V := V$ under $do(X{=}0)$, X factual	60.0 %	72.0 %

Each hypothesis is one rule for a whole question type, chosen on the tune half. Conditioning the observation in the post-intervention world agrees with the definition $Y_x \mid Z_x = z$.

7.7 Experiment 7: causal notations

Every CounterBench question was rewritten in 7 equation notations and 6 query notations ($do(\neg X) \to Y$, $P(Y{=}1 \mid do(X{=}0))$, $Y_{\neg X}$, $\neg X \mathrel{\square\!\!\to} Y$, $[do(\neg X)]\, Y$, $Y \mid do(X{=}0)$), giving 50,400 queries.

Equation notation	Accuracy under each of the 6 query notations
`V := e`	93.75 % (all 6)
`V` $:=$ `e`	93.75 % (all 6)
`V = e` (words: not / and / or)	93.75 % (all 6)
`V := f(e)`	93.75 % (all 6)
`V` $\leftrightarrow$ `e`	93.75 % (all 6)
`V <- e` (words)	93.75 % (all 6)
`e` $\to$ `V`	93.75 % (all 6)

All 42 combinations reach exactly the accuracy of the original template.

7.8 Experiment 8: unseen public logic data

Data	Router (final)	Closed world on all symbols	Forced open world
ProverQA easy (500)	100.00 %	94.80 %	100.00 %
ProverQA medium (500)	98.60 %	98.20 %	98.80 %
ProverQA hard (500)	99.60 %	99.60 %	99.60 %
ProverQA all (1,500)	**99.40 %**	97.53 %	99.47 %
FLD.v2 test (5,000)	96.58 %	–	98.08 %
ProofWriter OWA test (600)	84.50 %	83.50 %	99.50 %

ProverQA (Prover9-verified FOL), FLD (synthetic deduction in its own notation) and ProofWriter under the open world were used as an outside check; no parameter was tuned on them. Two of them exposed errors that were then fixed: ProverQA showed that questions about symbols absent from the KB must not be closed, and FLD showed that its notation was mis-parsed (69.88 % before support was added). After each fix the three development datasets were re-run and no answer

changed. ProofWriter OWA marks the limit of a structural router: its rule bases look exactly like RuleTaker's, but unprovable questions are Unknown there and true in RuleTaker, so no rule based on the input can satisfy both.

7.9 Experiment 9: generated test in 13 notations

Problem type (source of the gold label)	Problems	Accuracy
first-order logic, open world (z3)	112	100.00 %
propositional (truth table)	48	97.92 %
rule base, closed world (separate NAF evaluator)	64	100.00 %
causal (brute-force simulation)	40	100.00 %
all	**264**	**99.62 %**

A generator produced problems with nonsense names in 13 notations. Gold labels come from independent tools, not from the agent. The single error is a deep propositional problem that is not proved within the budget.

8 Error analysis

On FOLIO validation, raising the prover's budget does not change the errors, and none of the knowledge bases is contradictory. In eleven questions the conclusion uses a predicate that never occurs in the premises; most other errors come from mismatches between premise and conclusion annotations (`y1984` versus `1984`, `LostAt` versus `At`). Since FOLIO labels were written for the natural-language text, a sound prover over the annotations has a ceiling below 100 %; on clean FOL (ProverQA) the same prover reaches 99.40 %. Merging near-identical symbols inside the knowledge base was rejected: it gained only 0.2 points on train and risks merging opposite concepts.

9 Reasoning traces

The notebook prints a trace for every answer (Section 11 of the notebook). Five examples follow.

Forward chaining (assignment example).

```
KB    : Young(bob)   Kind(bob)
        ∀x (Young(x) → Smart(x))
        ∀x (Smart(x) ∧ Kind(x) → Successful(x))
Query : Successful(bob)
Answer: True   [forward chaining (closed world) | closure]
  Successful(bob)        [rule 3: Smart(x) ∧ Kind(x) → Successful(x)]
      Smart(bob)         [rule 2: Young(x) → Smart(x)]
          Young(bob)     [given fact]
      Kind(bob)          [given fact]
```

Negation as failure (RuleTaker, closed world).

```
KB    : Quiet(fiona)   Kind(bob)   Young(bob)
        ∀x (Kind(x) ∧ Young(x) → ¬Smart(x))
        ∀x (Quiet(x) ∧ ¬Smart(x) → Nice(x))
Query : Nice(fiona)
Answer: True   [forward chaining (closed world) | closure]
  Nice(fiona)            [rule 4: Quiet(x) ∧ ¬Smart(x) → Nice(x)]
      Quiet(fiona)       [given fact]
      ¬Smart(fiona)      [not derivable → true by negation as failure]
```

Resolution with reasoning by cases (open world).

```
KB    : ∀x (Drinks(x) → Dependent(x))     ∀x (Drinks(x) ⊕ Jokes(x))
        ∀x (Jokes(x) → ¬Unaware(x))       Unaware(rina)
Query : Dependent(rina)
Answer: True   [resolution (open world) | proved]
  ¬query  : {¬Dependent(rina)}
  resolve : {¬Dependent(rina)} + {Dependent(x0), ¬Drinks(x0)}  ⇒  {¬Drinks(rina)}
  resolve : {¬Drinks(rina)}    + {Drinks(x0), Jokes(x0)}       ⇒  {Jokes(rina)}
  resolve : {Jokes(rina)}      + {¬Jokes(x0), ¬Unaware(x0)}    ⇒  {¬Unaware(rina)}
  resolve : {¬Unaware(rina)}   + {Unaware(rina)}               ⇒  □  (empty clause)
```

A FOLIO v2 validation item.

```
KB    : ∀x (Employee(x) ∧ Schedule(x, meeting, customers) → AppearIn(x, company))
        ∀x (Employee(x) ∧ HasLunch(x, company) → Schedule(x, meeting, customers))
        ∀x (Employee(x) → (HasLunch(x, company) ⊕ HasLunch(x, home)))
        ∀x (Employee(x) ∧ HasLunch(x, home) → Work(x, home))
        ∀x (Employee(x) ∧ ¬In(x, homecountry) → Work(x, home))
        ∀x (Manager(x) → ¬Work(x, home))
        ¬(Manager(james) ⊕ AppearIn(james, company))
Query : Manager(james) → ¬Work(james, home)          (gold label: True)
Answer: True   [resolution (open world) | proved]
  ¬query  : {Work(james, home)}   {Manager(james)}
  resolve : {Work(james, home)} + {¬Manager(x0), ¬Work(x0, home)}  ⇒  {¬Manager(james)}
  resolve : {¬Manager(james)}   + {Manager(james)}                 ⇒  □  (empty clause)
```

CounterBench conditional query.

```
Equations : Blaf := ¬Ziklo    Vork := Blaf ∧ Trune    Lumbo := Vork
Observed  : Trune                 Query: Lumbo under do(¬Ziklo)
  1) abduction  : roots {Trune: False, Ziklo: True}   (weight 0.25)
  2) action     : do(Ziklo = False)   (its equation is cut)
  3) prediction : Blaf = True, Vork = False, Lumbo = False
                  contradicts the observation Trune: discarded
  1) abduction  : roots {Trune: True, Ziklo: True}    (weight 0.25)
  2) action     : do(Ziklo = False)
  3) prediction : Blaf = True, Vork = True, Lumbo = True
Answer: yes  (P = 1.0)
```

10 Limitations

Risk	Mitigation and status
Unknown notation in the private test	14 formula notations and 42 causal notation combinations tested (Experiments 4, 7, 9); strict-then-tolerant parsing; a broken formula is dropped, not the whole knowledge base. Not supported: DIMACS CNF and natural language.
Closed vs. open world chosen wrongly	Structural rule validated on all three datasets; Experiment 2 quantifies the cost of the wrong choice.
Open-world rule bases (ProofWriter style)	Indistinguishable from RuleTaker by structure; the closed world is kept (RuleTaker 100 %, ProofWriter OWA 84.5 %).
Annotation noise (FOLIO)	Guarded query alignment, tuned on the training split only.
Hard first-order problems	Deterministic budget of 50 given clauses; an unproved query is answered False.
Contradictory knowledge base	Set of support does not exploit the contradiction (classically every query would follow); this matches how the datasets label such stories.
Cyclic causal graph	Detected and reported with the cycle (CausalCycleError); answered False.
Notebook run without internet	A missing dataset only skips its evaluation; the test file is still answered and `submission.csv` is written.

Table 5: Known risks and how they are handled.

11 Conclusion

A single classical pipeline, made of a multi-dialect parser, three logic engines, a causal model and a structural router, answers all three problem families without a dataset flag. It reaches 99.63 % on the combined report splits and 99.33 % on the logic-only private test, and every answer comes with an explanation. The main open issue is the convention for open-world rule bases, which cannot be recognised from the input alone.

Reproducibility. The notebook `reasoning_agent.ipynb` runs top to bottom in Google Colab. It loads the public data, prints the exploration, the splits, the evaluation, the experiments (when `RUN_EXPERIMENTS = True`) and the traces, and writes `submission.csv`. A missing dataset only skips its evaluation; the test file is always answered.